CSA0210 – C PROGRAMMING

C01-AT3 Laboratory Evaluation

P Parvash khan
ID: 192312552

TASK 1 - Implement and Test the Billing Program

Objective: To implement a C program that calculates electricity bills using slab rates and tests normal bills, boundary cases, invalid inputs, meter rollover, and small billing examples.

1. Assumed Input:
Since the question does not specify exact tariff rates, the following rates are assumed for demonstration:

Units consumed	Rate per unit
First 100 units	₹2
Next 100 units	₹3
Above 200 units	₹5

- Meter readings range from 0 to 65,535.
- If the current reading is less than the previous reading, assume a meter rollover.
- Negative readings and non-numeric inputs are invalid.

2. C Program Implementation:

```
#include <stdio.h>
#include <limits.h>

long long calculateBill(long long units) {
    long long bill = 0;
    if (units > 200) {
        bill += (units - 200) * 500;
        units = 200;
    }
    if (units > 100) {
        bill += (units - 100) * 300;
        units = 100;
    }
    bill += units * 200;
    return bill;
}

int main(void) {
    long long previous, current, units;
    printf("Enter previous meter reading: ");
    if (scanf("%lld", &previous) != 1)
        return printf("Invalid input\n"), 1;

    printf("Enter current meter reading: ");
    if (scanf("%lld", &current) != 1)
        return printf("Invalid input\n"), 1;

    if (previous < 0 || previous > 65535 || current < 0 || current > 65535) {
        printf("Invalid meter reading\n");
        return 1;
    }

    if (current >= previous) {
        units = current - previous;
    } else if (previous >= 65000 && current <= 100) {
        units = 65536 - previous + current;
    } else {
        printf("Invalid reading: no valid meter rollover\n");
        return 1;
```

3. Assumed Test Inputs and Expected Results:

Test Case	Previous	Current	Expected Result
Normal bill	100	150	50 units, ₹100
Boundary case	0	100	100 units, ₹200
Boundary case	100	200	100 units, ₹200
Zero units	500	500	0 units, ₹0
Meter rollover	65535	0	1 unit, ₹2
Invalid negative reading	-1	50	Invalid meter reading
Invalid reading order	500	400	Invalid reading
Non-numeric input	abc	-	Invalid input

Note: All tariff rates are assumed. The program stores the bill internally in paise, using integer arithmetic to avoid floating-point rounding errors.

4. Alternative Concept and Complexity Analysis:

If slab limits and rates are stored in arrays, a loop can calculate the bill instead of using fixed if statements.

- **Time Complexity:** O(s) where *s* is the number of tariff slabs. With a fixed number of slabs (e.g., three), time complexity reduces to **O(1)**.
- **Extra Space:** O(s) auxiliary space for storing slab arrays.

This approach is significantly more flexible because new tariff slabs can be appended or updated dynamically without altering underlying calculation logic.

TASK 2 - Compare Expectations with Experiment (4 Marks)

Bloom's Level: K4 - Analyse

Objective: To compare floating-point calculations with integer-paise calculations and analyse precision, overflow, and memory usage.

Assumed Inputs:
Test total bill amounts: ₹4,000 | ₹60,000 | ₹8,00,000

C Program:

```
#include <stdio.h>

int main(void) {
    long long amounts[] = {4000, 60000, 800000};
    int n = 3;

    printf("Amount (Rs.)\tFloat (Rs.)\tPaise (Rs.)\n");
    for (int i = 0; i < n; i++) {
        float total = (float)amounts[i];
        long long paise = amounts[i] * 100LL;
        printf("%lld\t\t%.2f\t\t%.2f\n", amounts[i], total, paise / 100.0);
    }
    return 0;
}
```

Expected Results:

Bill Amount	Float Result	Paise Result
₹4,000	₹4,000.00	₹4,000.00
₹60,000	₹60,000.00	₹60,000.00
₹8,00,000	₹8,00,000.00	₹8,00,000.00

The results agree for these whole-rupee values. However, floating-point arithmetic cannot represent every decimal value exactly, while integer paise provides exact arithmetic for amounts expressed in whole paise, provided the integer does not overflow.

Alternative Concept & Complexity Analysis:

If total is stored in a double instead of a long long integer, very large values can be represented, but decimal calculations may still suffer rounding errors. Integer paise is preferable for financial applications requiring predictable exactness.

- **Time Complexity:** O(n) where *n* is the number of transaction amounts.
- **Extra Space:** O(1) auxiliary space, excluding input data storage.

TASK 3 - Recommend and Defend the Final Design (4 Marks)

Bloom's Level: K5 - Evaluate

Objective: To recommend a reliable billing program using appropriate data types, constants, and preprocessor directives. Reads rates at runtime to support dynamic tariff updates without recompilation.

C Program:

```
#include <stdio.h>

int main(void) {
    long long rate1, rate2, rate3;
    long long units, bill = 0;

    FILE *fp = fopen("tariff.txt", "r");
    if (fp == NULL) {
        printf("Cannot open tariff file\n");
        return 1;
    }

    if (fscanf(fp, "%lld %lld %lld", &rate1, &rate2, &rate3) != 3) {
        printf("Invalid tariff configuration\n");
        fclose(fp);
        return 1;
    }
    fclose(fp);

    printf("Enter units consumed: ");
    if (scanf("%lld", &units) != 1 || units < 0) {
        printf("Invalid input\n");
        return 1;
    }

    if (units <= 100)
        bill = units * rate1;
    else if (units <= 200)
        bill = 100 * rate1 + (units - 100) * rate2;
    else
        bill = 100 * rate1 + 100 * rate2 + (units - 200) * rate3;

    printf("Units: %lld\n", units);
    printf("Total bill: Rs. %.2f\n", bill / 100.0);
```

DAY 4 - DESIGN CHALLENGE: TASK 4 - Re-evaluate and Adapt Design (8 Marks)

Bloom's Level: K5 - Evaluate

1. Changed Requirement:
The electricity cooperative installs smart meters that send readings every 15 minutes. The tariff charges a 20% surcharge for electricity consumed during peak hours between 6:00 PM and 10:00 PM. Dynamic runtime tariff loading must be maintained.

2. Assumed Input & Metrics:

- Readings per hour: 4 | Readings per day: 24 × 4 = 96
- Readings per year: 96 × 365 = 35,040 intervals
- Peak Period: 6:00 PM to 10:00 PM (Hours 18 to 21)
- Normal Tariff: ₹2.00 / unit | Peak Tariff (+20%): ₹2.40 / unit
- Consumption format: milli-units (1,000 milli-units = 1 unit)

3. Adapted C Program:

```
#include <stdio.h>

#define READINGS_PER_DAY 96
#define DAYS_PER_YEAR 365

int main(void) {
    FILE *fp, *config;
    long long normalRate, surcharge;
    long long milliUnits, totalMilliUnits = 0;
    long double totalPaise = 0;
    int i, hour;

    config = fopen("tariff_config.txt", "r");
    if (config == NULL) {
        printf("Cannot open tariff configuration\n");
        return 1;
    }

    if (fscanf(config, "%lld %lld", &normalRate, &surcharge) != 2 || normalRate < 0 ||
surcharge < 0) {
        printf("Invalid tariff configuration\n");
        fclose(config);
        return 1;
    }
    fclose(config);

    fp = fopen("readings.txt", "r");
    if (fp == NULL) {
        printf("Cannot open readings file\n");
        return 1;
    }

    for (i = 0; i < READINGS_PER_DAY * DAYS_PER_YEAR; i++) {
        if (fscanf(fp, "%lld", &milliUnits) != 1 || milliUnits < 0) {
            printf("Invalid or missing interval reading\n");
            fclose(fp);
            return 1;
        }

        hour = (i / 4) % 24;
        totalMilliUnits += milliUnits;

        if (hour >= 18 && hour < 22)
            totalPaise += (long double)milliUnits * normalRate * (100 + surcharge) / 100.0L;
        else
            totalPaise += (long double)milliUnits * normalRate;
    }
```

4. Expected Time-based Behavior:

Time Period	Base Rate	Applied Rate
10:00 AM	₹2.00 / unit	₹2.00 / unit
5:45 PM	₹2.00 / unit	₹2.00 / unit
6:00 PM	₹2.00 / unit	₹2.40 / unit
9:45 PM	₹2.00 / unit	₹2.40 / unit
10:00 PM	₹2.00 / unit	₹2.00 / unit

5. Re-evaluation Matrix (Original vs Modified Design):

Original Design	Modified Smart-Meter Design
Uses cumulative meter readings	Uses 15-minute interval consumption readings
Applies fixed slab rates	Applies time-of-use peak surcharge rates
Hard-coded logic / static structures	Tariff settings loaded dynamically from file
Calculates basic single-period bill	Calculates annual time-differentiated bill
Suitable for legacy electromechanical meters	Suitable for modern IoT smart-meter infrastructure

6. Complexity Analysis:

- **Time Complexity:** O(n), where *n* is the total number of 15-minute readings. Processing 35,040 annual readings takes negligible execution time.
- **Space Complexity:** O(1) auxiliary space because readings are processed sequentially in a stream without storing historical arrays in memory. Over 15 years (525,600 readings), memory overhead remains constant.

7. Final Conclusion & Future Recommendations:
The modified design efficiently scales to smart meter time-of-use billing requirements while keeping memory usage at O(1). For enterprise deployment, the system should incorporate checksum verification for timestamp integrity, handle clock synchronization / daylight savings adjustments, and guard against overflow in large multi-year calculations.